Filho paga para o Pai

Lucas Panonko Assis Belchior - Matrícula 12311BCC068

Detecção e análise de candidatos a Child Pays for Parent (CPFP)

1. Objetivo e dados

O objetivo é detectar uma transação pai de baixa taxa e uma transação filho que consome pelo menos uma saída do pai, paga uma taxa maior e aparece no mesmo bloco. As taxas das duas transações são destinadas ao minerador que produziu esse bloco. A classificação econômica é uma heurística: a inclusão conjunta comprova a dependência, mas não a intenção de acelerar a confirmação [1, 2].

Característica da amostra	Resultado
Fonte	blocos.tgz fornecido nos exercícios da disciplina
Arquivo compactado	207.436.070 bytes
Blocos analisados	98
Alturas mínima e máxima	960179 a 960802
Período dos timestamps (UTC)	2026-07-30T00:39:11+00:00 até 2026-08-03T01:46:10+00:00
Transações normais / coinbase	493.473 / 98
Anomalias de integridade detectadas	0

A amostra não é tratada como uma sequência contínua de blocos nem como representativa de toda a rede. Os JSONs são lidos diretamente do TAR compactado, um bloco por vez. O hash do arquivo e os comandos de reprodução estão na página de referências.

Resultado principal: 552 pares candidatos, em 60 dos 98 blocos.

Como o algoritmo detecta os pares

2. Dependência e unidade de análise

Cada saída é identificada por (tx_index, n), no formato fornecido. O tx_index é um identificador do provedor; o hash da transação é conservado para auditoria. Uma entrada do filho deve referenciar uma saída existente do pai, com o mesmo valor, e o pai deve aparecer antes do filho no bloco. Várias saídas consumidas do mesmo pai formam uma única aresta.

Construímos um grafo por bloco. A análise econômica usa somente componentes conexos com duas transações e uma aresta: um par simples e isolado. Componentes com três ou mais transações são contados separadamente. Isso evita atribuir a um par a taxa de um pacote que contém outros ancestrais.

3. Taxas e regra principal

```
f = soma das entradas - soma das saídas (satoshis)
v = teto(peso / 4) (vbytes); r = f / v (sat/vB)
r_conjunto = (f_pai + f_filho) / (v_pai + v_filho)
```

M é a mediana de sat/vB das transações normais do próprio bloco. Um par simples é candidato quando atende simultaneamente:

Condição	Interpretação
r_pai < M	Pai relativamente barato no bloco
f_filho > f_pai	Filho paga mais em taxa absoluta
r_filho > r_pai e r_filho >= 2 × r_pai	Filho tem maior taxa por tamanho
r_conjunto >= M	Conjunto alcança a referência adotada

Pai com taxa zero é permitido; o filho deve pagar taxa positiva. Não se calcula razão dividindo por zero. Se M = 0, nenhum par atende à primeira condição. M não é o limite de aceitação do minerador: é uma escolha didática.

Fluxo: ler e validar bloco → indexar transações/saídas → criar arestas → identificar componentes → aplicar regra → agregar resultados. O grafo custa O(T + I + O), em média, por bloco; a mediana e as ordenações acrescentam custo. T, I e O representam transações, entradas e saídas.

Frequência e comparação das taxas

Indicador	Quantidade
Dependências diretas no mesmo bloco	338.781
Pares simples elegíveis	8.375
Componentes complexos / transações nesses componentes	18.227 / 346.189
Candidatos / pares simples	552 / 8.375 = 6,591%
Filhos candidatos / transações normais	552 / 493.473 = 0,112%

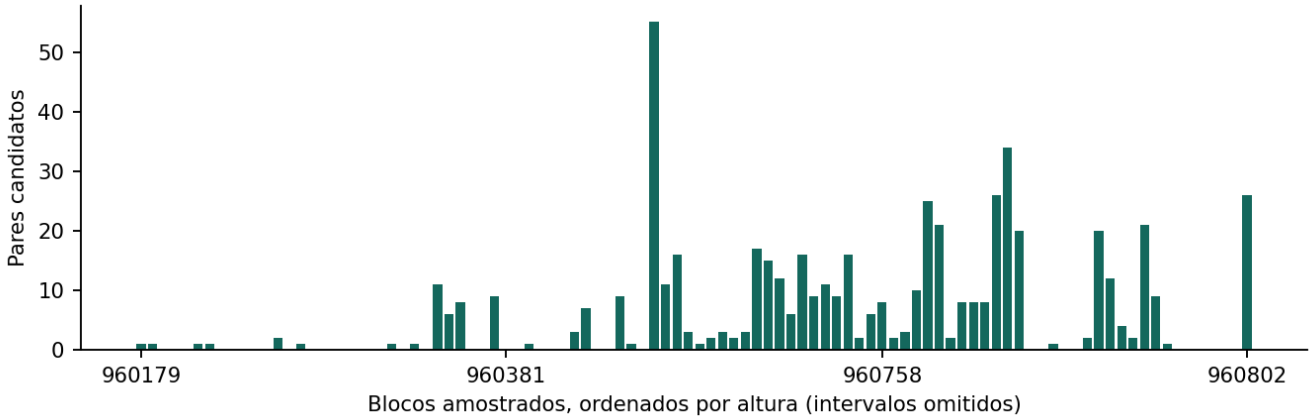


Figura 1. Candidatos por bloco amostrado; os intervalos sem blocos no arquivo não são preenchidos com zeros.

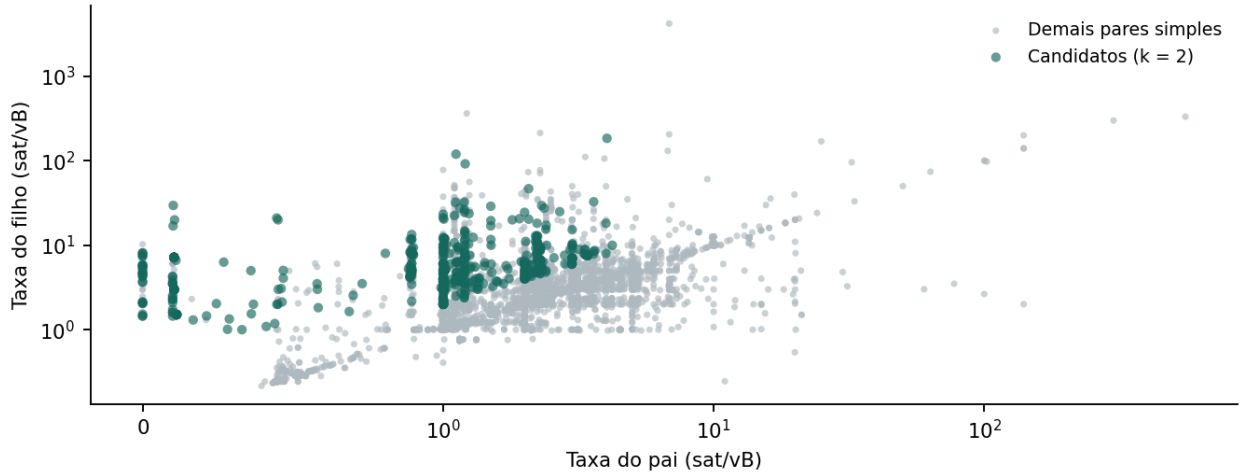


Figura 2. Todos os pares simples. Escala symlog nos dois eixos: linear até 1 sat/vB e logarítmica acima, preservando os casos de taxa zero. Os pontos destacados satisfazem todas as condições, não apenas a diferença de taxas.

Taxas, valores e endereços

Medida entre candidatos	Mínimo	Mediana	Máximo
Taxa do pai (sat)	0	170,0	5.051
Taxa do filho (sat)	111	822,0	35.523
Pai (sat/vB)	0,000	1,007	4,220
Filho (sat/vB)	1,000	5,028	184,734
Conjunto (sat/vB)	0,561	3,002	68,448
Valor consumido do pai (sat)	330	164.939,5	13.132.720.455

Os candidatos têm 552 pais e 552 filhos distintos; 26 pais pagam taxa zero. As taxas das transações únicas somam 841.115 sat. O valor das saídas dos pais consumidas pelos filhos soma 68.388.461.156 sat (683,88461156 BTC). Esse valor pode circular em outras transações: não representa dinheiro novo ou prejuízo.

Foram observados 469 endereços distintos nas saídas consumidas dos pais; 0 de 572 saídas não possuem endereço informado.

Endereço da saída do pai	Pares	Valor (sat)
bc1pfeessrawgf	57	27.018
bc1q0h8mujmkue3yvfwdg5dhqvgcpmmse050anch0r	12	3.708
bc1qns9f7yfx3ry9lj6yz7c9er0vwa0ye2eklpzqfw	11	2.949.979.608
bc1pp0aue59r0shhzrc0ph9l6an8d96utq4gmk9hc2n5g60vvz0rv4ssnux995	8	2.640
bc1pa4yf6uz25jukh7hvdn22q5y2a6q9ufrw9pnv9lf8xhj9j9prhyfs4e0pmr	5	5.000

Tabela de até cinco endereços mais recorrentes. Um endereço é contado uma vez por par, mesmo se houver várias saídas consumidas. Os valores agregam essas saídas. Endereço não equivale a pessoa; a análise não identifica proprietários nem deduz que pai e filho pertencem à mesma carteira.

Exemplos rastreáveis nos dados

Exemplo 1 - bloco 960793

```
Pai: 6cf4de366f95dd7d85f4a8bd4082dbc22409f376e5e98416875869def860323f
Filho: 00a84f00d655d067144dae372916620b62df0969487f57e86e10a5d46e9aa0f0
```

Medida	Pai	Filho
Taxa (sat)	164	23.040
Tamanho virtual (vB)	147	192
Taxa por tamanho (sat/vB)	1,1156	120,0000

Taxa conjunta: 68,4484 sat/vB; mediana do bloco: 2,0120 sat/vB. Aumento absoluto em relação ao pai: 67,3327 sat/vB. Valor consumido: 111.271 sat. Classificação: candidato.

saída 0: 111.271 sat, 16PukMgi3ovHb5QuZSM85zuKg8r8FdqHRR

Exemplo 2 - bloco 960179

```
Pai: 5da18bca08910617e0be0f9e4343ec477ef1a30b268df9ace1feab062f1813ac
Filho: 562f9f09951c152bc580c10e4ba96ce06b59b2d580f1181e891821a610731d0d
```

Medida	Pai	Filho
Taxa (sat)	0	503
Tamanho virtual (vB)	155	347
Taxa por tamanho (sat/vB)	0,0000	1,4496

Taxa conjunta: 1,0020 sat/vB; mediana do bloco: 0,3500 sat/vB. Aumento absoluto em relação ao pai: 1,0020 sat/vB. Valor consumido: 2.852.000 sat. Classificação: candidato.

saída 0: 2.852.000 sat, bc1qadwv77kqm0n6h92jw6mar9a24r8vlsdc7aj2v3l2l3wyheyxefcsjq0pf4

Seleção dos exemplos: maior aumento absoluto da taxa do conjunto em relação ao pai e, depois, menor taxa do pai entre os demais candidatos. Os vínculos e cálculos foram reconferidos diretamente nos blocos originais. A seleção ilustra o mecanismo e não representa o caso típico.

Sensibilidade, limites e conclusão

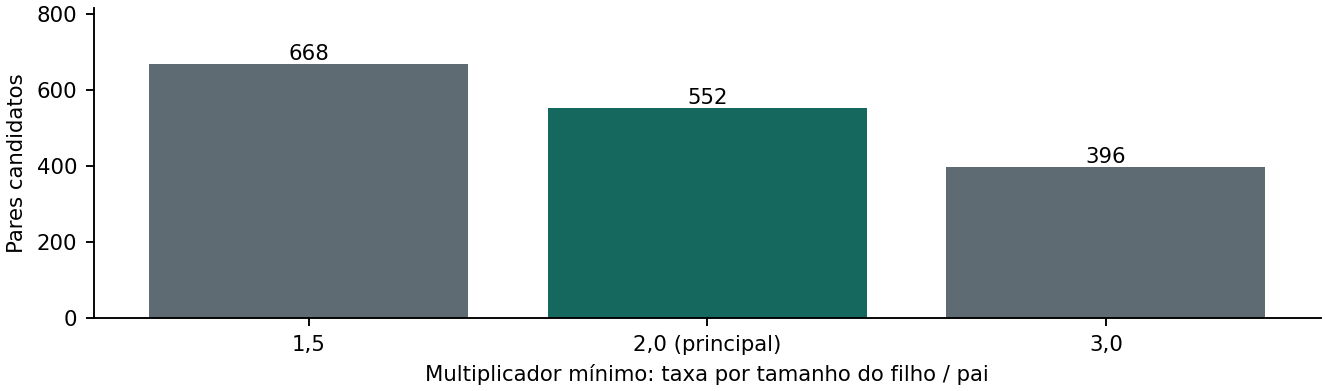


Figura 3. Variação apenas do multiplicador mínimo da taxa por tamanho do filho em relação ao pai. Todas as demais condições permanecem fixas. Os três valores foram definidos antes de examinar os resultados.

Limitações

A regra usa taxas relativas ao próprio bloco, sem estimar a política real do minerador. Não há dados históricos da mempool, tempo de espera, declaração de intenção ou rótulos verdadeiros. Por isso, não se calculam acurácia, precisão ou recall, e os pares classificados são candidatos compatíveis com CPFP, não ocorrências confirmadas por intenção.

Pacotes com três ou mais transações ficam fora da classificação principal, embora possam conter CPFP. A contagem é restrita aos pares simples da amostra e não mede a frequência global da técnica. Gastos comuns podem satisfazer os critérios. Endereços ausentes limitam a análise de recorrência.

Validação

Os testes sintéticos verificam: taxa zero, mediana zero, arredondamento de peso, taxas inconsistentes, entradas repetidas do mesmo pai, pais externos, cadeias, ramificações, coinbase, referências internas inválidas e endereço ausente. Na base completa, foram conferidos duplicados, taxas, referências internas e a ordem pai-filho. Os dois exemplos foram recalculados na fonte.

Conclusão

O algoritmo encontrou 552 candidatos entre 8.375 pares simples (6,591%). O trabalho mostra como o grafo de UTxOs e a taxa conjunta permitem detectar um padrão econômico explicável. A principal distinção aprendida é entre observar uma dependência na cadeia e comprovar a motivação de quem criou a transação.

Reprodução e referências

Código e resultados

https://github.com/LuksbellN/ESP_SEC-Filho-paga-para-o-Pai

O repositório contém os dois scripts, testes, tabelas resumidas e este PDF. O arquivo original não é redistribuído: usar o blocos.tgz fornecido pela disciplina. O apêndice traz o código integral, para que o documento seja autossuficiente. As tabelas completas são regeneradas pelos comandos abaixo.

Python 3.12 ou superior. Dentro da pasta do repositório, com um interpretador Python instalado e funcional:

Dependências usadas: matplotlib 3.10.7, reportlab 4.4.9 e pypdf 6.10.0 (conteúdo de requirements.txt). O detector usa somente a biblioteca padrão.

```
python -m pip install -r requirements.txt
python -m unittest -v test_analyze
python analyze.py --archive ../blocos.tgz --output results
python build_report.py --results results
No último comando, o PDF é gravado em report/.
```

Identificação exata do arquivo de entrada (SHA-256):

```
679fe208f9a807fdd38e9d64948c73581af3e99ca52fb707ce06937e67bc536d
```

Saídas: pairs.csv (pares simples e motivos de rejeição), blocks.csv (censo por bloco), addresses.csv (endereços dos candidatos) e summary.json (parâmetros, totais, métricas e exemplos). Valores monetários estão em satoshis; 1 BTC = 100.000.000 sat.

Referências

[1] Enunciado da atividade “(P1) Filho paga para o Pai”, disciplina de Tópicos Especiais de Segurança. Material fornecido ao aluno, 2026.

[2] BITCOIN CORE. Bitcoin Core 0.13.0 Released!, seção “More intelligent transaction selection for mining”, 23 ago. 2016. Disponível em: https://bitcoincore.org/en/2016/08/23/release-0.13.0/. Acesso em: 2 nov. 2026.

[3] blocos.tgz. Conjunto de blocos JSON fornecido nos exercícios anteriores da disciplina. Arquivo identificado pelo SHA-256 acima.

Apêndice - código-fonte integral

As páginas seguintes reproduzem analyze.py e build_report.py com números de linha. Linhas longas são quebradas visualmente; para execução, usar os arquivos do repositório. A lógica de detecção não depende de rede, serviços pagos ou aprendizado de máquina.

Apêndice | analyze.py

```
"""Detector didático de candidatos a CPFP em blocos JSON do arquivo da disciplina."""
import argparse
import csv
import hashlib
import json
import math
import platform
import statistics
import tarfile
from collections import defaultdict
from datetime import datetime, timezone
from pathlib import Path

KS = (1.5, 2.0, 3.0)


def coinbase(tx):
    ins = tx['inputs']
    return len(ins) == 1 and ins[0]['prev_out'].get('n') == 4294967295


def transaction_metrics(tx: dict) -> dict:
    """Valores em satoshis; tamanho virtual conforme peso, nunca tamanho bruto."""
    ins, outs = tx['inputs'], tx['out']
    if not ins or not outs:
        raise ValueError('Transação sem entradas ou saídas')
    values = [i['prev_out']['value'] for i in ins] + [o['value'] for o in outs]
    if any(type(v) is not int or v < 0 for v in values):
        raise ValueError('Valor monetário inválido')
    fee = sum(i['prev_out']['value'] for i in ins) - sum(o['value'] for o in outs)
    weight = tx['weight']
    if fee < 0 or tx.get('fee', fee) != fee:
        raise ValueError('Taxa incompatível com entradas e saídas')
    if type(weight) is not int or weight <= 0:
        raise ValueError('Peso inválido')
    vsize = (weight + 3) // 4
    return dict(fee_sat=fee, vsize=vsize, rate=fee / vsize)


def rejection_reasons(fp, vp, fc, vc, median_rate, k=2.0):
    rp, rc = fp / vp, fc / vc
    conditions = (
        ('pai_abaixo_mediana', rp < median_rate),
        ('filho_taxa_absoluta_maior', fc > fp),
        ('filho_taxa_relativa_maior', rc > rp and rc >= k * rp),
        ('conjunto_alcanca_mediana',
         (fp + fc) / (vp + vc) >= median_rate),
    )
    return [name for name, valid in conditions if not valid]


def is_candidate(fp: int, vp: int, fc: int, vc: int,
                 median_rate: float, k: float = 2.0) -> bool:
    return not rejection_reasons(fp, vp, fc, vc, median_rate, k)


def analyze_block(block: dict) -> dict:
    txs = block['tx']
    if block['n_tx'] != len(txs):
        raise ValueError('Contagem de transações inconsistente')
    index, positions, metrics, outputs = {}, {}, {}, {}
    hashes = set()
    for pos, tx in enumerate(txs):
        idx = tx['tx_index']
        if idx in index or tx['hash'] in hashes:
```

Apêndice | analyze.py

```
            raise ValueError('Identificador de transação duplicado')
        index[idx], positions[idx] = tx, pos
        hashes.add(tx['hash'])
        outmap = {o['n']: o for o in tx['out']}
        if len(outmap) != len(tx['out']):
            raise ValueError('Índice de saída duplicado')
        outputs[idx] = outmap
        if not coinbase(tx):
            metrics[idx] = transaction_metrics(tx)
    median = statistics.median(m['rate'] for m in metrics.values()) if metrics else 0
    edges, adjacent, spent = {}, defaultdict(set), set()
    for child, tx in index.items():
        if child not in metrics:
            continue
        for item in tx['inputs']:
            prev = item['prev_out']
            parent, n = prev['tx_index'], prev['n']
            key = parent, n
            if key in spent:
                raise ValueError('Saída consumida mais de uma vez no bloco')
            spent.add(key)
            if parent not in index:
                continue
            if parent not in metrics or positions[parent] >= positions[child]:
                raise ValueError('Pai interno inválido ou fora de ordem')
            output = outputs[parent].get(n)
            if output is None or output['value'] != prev['value']:
                raise ValueError('Saída interna inexistente ou valor divergente')
            edges.setdefault((parent, child), []).append(dict(
                n=n, value=output['value'], address=output.get('addr')))
            adjacent[parent].add(child)
            adjacent[child].add(parent)
    visited, simple, complex_sizes = set(), [], []
    for start in adjacent:
        if start in visited:
            continue
        stack, component = [start], set()
        visited.add(start)
        while stack:
            node = stack.pop()
            component.add(node)
            for neighbor in adjacent[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    stack.append(neighbor)
        if len(component) == 2:
            a, b = sorted(component, key=positions.get)
            simple.append((a, b))
        else:
            complex_sizes.append(len(component))
    pairs = []
    for parent, child in simple:
        p, c = metrics[parent], metrics[child]
        fp, vp, fc, vc = p['fee_sat'], p['vsize'], c['fee_sat'], c['vsize']
        row = dict(height=block['height'], parent=index[parent]['hash'],
                   child=index[child]['hash'], parent_fee=fp, child_fee=fc,
                   parent_vsize=vp, child_vsize=vc, parent_rate=p['rate'],
                   child_rate=c['rate'], median_rate=median,
                   package_rate=(fp + fc) / (vp + vc),
                   consumed_sat=sum(o['value'] for o in edges[parent, child]),
                   consumed_outputs=edges[parent, child])
        row['uplift'] = row['package_rate'] - row['parent_rate']
        for k in KS:
            row[f'candidate_{k}'] = is_candidate(fp, vp, fc, vc, median, k)
        row['reasons'] = '|'.join(rejection_reasons(fp, vp, fc, vc, median))
```

Apêndice | analyze.py

```
        pairs.append(row)
    info = dict(height=block['height'], hash=block['hash'], time=block['time'],
                normal_transactions=len(metrics), coinbase=len(txs)-len(metrics),
                median_rate=median, edges=len(edges), simple_pairs=len(pairs),
                complex_components=len(complex_sizes),
                complex_transactions=sum(complex_sizes),
                candidates=sum(p['candidate_2.0'] for p in pairs))
    return dict(block_row=info, pairs=pairs, complex_components=complex_sizes)


def write_csv(path, rows, fields):
    with path.open('w', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        for row in rows:
            writer.writerow({k: json.dumps(v, ensure_ascii=False) if isinstance(v, list)
                             else v for k, v in row.items()})


def stats(values):
    return dict(count=len(values), minimum=min(values) if values else None,
                median=statistics.median(values) if values else None,
                maximum=max(values) if values else None, total=sum(values))


def run_analysis(archive: Path, output: Path) -> dict:
    output.mkdir(parents=True, exist_ok=True)
    with archive.open('rb') as f:
        digest = hashlib.file_digest(f, 'sha256').hexdigest()
    blocks, pairs, seen_blocks, seen_heights, seen_txs = [], [], set(), set(), set()
    with tarfile.open(archive, 'r|gz') as tar:
        for member in tar:
            if not member.isfile():
                continue
            block = json.load(tar.extractfile(member))
            if block['hash'] in seen_blocks or block['height'] in seen_heights:
                raise ValueError('Bloco duplicado')
            if Path(member.name).name != block['hash']:
                raise ValueError('Nome do arquivo não corresponde ao hash do bloco')
            seen_blocks.add(block['hash'])
            seen_heights.add(block['height'])
            for tx in block['tx']:
                if tx['hash'] in seen_txs:
                    raise ValueError('Transação repetida entre blocos')
                seen_txs.add(tx['hash'])
            result = analyze_block(block)
            blocks.append(result['block_row'])
            pairs.extend(result['pairs'])
            print(f"Bloco {block['height']}: {len(result['pairs'])} pares; "
                  f"{result['block_row']['candidates']} candidatos", flush=True)
    if not blocks:
        raise ValueError('Arquivo sem blocos')
    blocks.sort(key=lambda b: b['height'])
    pairs.sort(key=lambda p: (p['height'], p['parent']))
    candidates = [p for p in pairs if p['candidate_2.0']]
    address_rows = {}
    for pair in candidates:
        by_address = defaultdict(int)
        for out in pair['consumed_outputs']:
            by_address[out['address'] or '(ausente)'] += out['value']
        for address, value in by_address.items():
            a = address_rows.setdefault(address, dict(address=address, pairs=0, value_sat=0))
            a['pairs'] += 1
            a['value_sat'] += value
    addresses = sorted(address_rows.values(), key=lambda a: (-a['pairs'], a['address']))
```

Apêndice | analyze.py

```
    first = sorted(candidates, key=lambda p: (-p['uplift'], p['height'], p['parent']))[:1]
    rest = sorted([p for p in candidates if p not in first],
                  key=lambda p: (p['parent_rate'], p['height'], p['parent']))[:1]
    examples = first + rest if candidates else pairs[:1]
    normal = sum(b['normal_transactions'] for b in blocks)
    metrics = ['parent_fee', 'child_fee', 'parent_rate', 'child_rate',
               'package_rate', 'consumed_sat', 'uplift']
    summary = dict(
        archive_sha256=digest, archive_bytes=archive.stat().st_size,
        python=platform.python_version(), parameters=dict(k=2.0, sensitivity=list(KS)),
        blocks=len(blocks), height_min=blocks[0]['height'], height_max=blocks[-1]['height'],
        start_utc=datetime.fromtimestamp(min(b['time'] for b in blocks), timezone.utc).isoformat(),
        end_utc=datetime.fromtimestamp(max(b['time'] for b in blocks), timezone.utc).isoformat(),
        normal_transactions=normal, coinbase=sum(b['coinbase'] for b in blocks),
        direct_dependencies=sum(b['edges'] for b in blocks), simple_pairs=len(pairs),
        complex_components=sum(b['complex_components'] for b in blocks),
        complex_transactions=sum(b['complex_transactions'] for b in blocks),
        candidates=len(candidates), unique_parents=len({p['parent'] for p in candidates}),
        unique_children=len({p['child'] for p in candidates}),
        blocks_with_candidates=sum(b['candidates'] > 0 for b in blocks),
        candidate_fraction_pairs=len(candidates)/len(pairs) if pairs else None,
        candidate_fraction_transactions=len(candidates)/normal if normal else None,
        zero_fee_parents=sum(p['parent_fee'] == 0 for p in candidates),
        sensitivity={str(k): sum(p[f'candidate_{k}'] for p in pairs) for k in KS},
        metrics={name: stats([p[name] for p in candidates]) for name in metrics},
        unique_addresses=sum(a['address'] != '(ausente)' for a in addresses),
        missing_address_outputs=sum(o['address'] is None for p in candidates
                                    for o in p['consumed_outputs']),
        consumed_outputs=sum(len(p['consumed_outputs']) for p in candidates),
        top_addresses=addresses[:5], examples=examples, anomalies=0,
    )
    # Componentes simples são disjuntos: taxas e valores não duplicam transações.
    summary['unique_fee_total_sat'] = sum(p['parent_fee'] + p['child_fee'] for p in candidates)
    fields = list(pairs[0]) if pairs else [
        'height', 'parent', 'child', 'parent_fee', 'child_fee', 'parent_vsize',
        'child_vsize', 'parent_rate', 'child_rate', 'median_rate', 'package_rate',
        'consumed_sat', 'consumed_outputs', 'uplift', 'candidate_1.5',
        'candidate_2.0', 'candidate_3.0', 'reasons']
    write_csv(output/'pairs.csv', pairs, fields)
    write_csv(output/'blocks.csv', blocks, list(blocks[0]))
    write_csv(output/'addresses.csv', addresses, ['address', 'pairs', 'value_sat'])
    (output/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
    print(json.dumps({k: summary[k] for k in ['blocks', 'normal_transactions',
          'direct_dependencies', 'simple_pairs', 'complex_components', 'candidates',
'sensitivity']}, indent=2))
    return summary


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--archive', type=Path, default=Path('../blocos.tgz'))
    parser.add_argument('--output', type=Path, default=Path('results'))
    args = parser.parse_args()
    run_analysis(args.archive, args.output)
```

Apêndice | build_report.py

```
"""Gera gráficos e PDF autossuficiente a partir dos resultados do analisador."""
import argparse
import csv
import json
import textwrap
from pathlib import Path
from xml.sax.saxutils import escape

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from reportlab.pdfgen import canvas
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle
from reportlab.platypus import Paragraph, Table, TableStyle

ROOT = Path(__file__).resolve().parent
REPO = 'https://github.com/LuksbellN/ESP_SEC-Filho-paga-para-o-Pai'
W, H = A4
M, CW = 43, A4[0] - 86
INK, TEAL, GRAY = '#202b36', '#14685d', '#5f6b73'


def n(value, digits=0):
    if value is None:
        return 'não aplicável'
    return f'{value:,.{digits}f}'.replace(',', '_').replace('.', ',').replace('_', '.')


def pct(value):
    return 'não aplicável' if value is None else n(value * 100, 3) + '%'


def plots(results, summary):
    figures = results / 'figures'
    figures.mkdir(exist_ok=True)
    with (results/'blocks.csv').open(encoding='utf-8') as f:
        blocks = list(csv.DictReader(f))
    with (results/'pairs.csv').open(encoding='utf-8') as f:
        pairs = list(csv.DictReader(f))
    plt.rcParams.update({'font.size': 9, 'axes.spines.top': False,
                         'axes.spines.right': False, 'figure.dpi': 170})
    fig, ax = plt.subplots(figsize=(8, 2.7))
    ax.bar(range(len(blocks)), [int(b['candidates']) for b in blocks], color=TEAL)
    ticks = sorted(set([0, len(blocks)//3, 2*len(blocks)//3, len(blocks)-1]))
    ax.set_xticks(ticks, [blocks[i]['height'] for i in ticks])
    ax.set(xlabel='Blocos amostrados, ordenados por altura (intervalos omitidos)',
           ylabel='Pares candidatos')
    fig.tight_layout(); fig.savefig(figures/'frequency.png'); plt.close(fig)
    fig, ax = plt.subplots(figsize=(8, 3.2))
    for flag, label, color, size in [(False, 'Demais pares simples', '#aeb9bf', 9),
                                     (True, 'Candidatos (k = 2)', TEAL, 19)]:
        rows = [p for p in pairs if (p['candidate_2.0'] == 'True') == flag]
        ax.scatter([float(p['parent_rate']) for p in rows],
                   [float(p['child_rate']) for p in rows], s=size,
                   c=color, label=label, alpha=.65, edgecolors='none')
    # A escala simétrica logarítmica mantém os zeros e torna os extremos visíveis.
    ax.set_xscale('symlog', linthresh=1)
    ax.set_yscale('symlog', linthresh=1)
    ax.set(xlabel='Taxa do pai (sat/vB)', ylabel='Taxa do filho (sat/vB)')
    ax.legend(frameon=False, fontsize=8)
    fig.tight_layout(); fig.savefig(figures/'rates.png'); plt.close(fig)
```

Apêndice | build_report.py

```
066      fig, ax = plt.subplots(figsize=(8, 2.5))
067      vals = list(summary['sensitivity'].values())
068      ax.bar(['1,5', '2,0 (principal)', '3,0'], vals, color=[GRAY, TEAL, GRAY])
069      for i, v in enumerate(vals): ax.text(i, v, n(v), ha='center', va='bottom')
070      ax.set(xlabel='Multiplicador mínimo: taxa por tamanho do filho / pai',
071             ylabel='Pares candidatos', ylim=(0, max(vals + [1])*1.22))
072      fig.tight_layout(); fig.savefig(figures/'sensitivity.png'); plt.close(fig)
073
074
075  class Report:
076      def __init__(self, dest):
077          fontdir = Path(matplotlib.get_data_path())/'fonts/ttf'
078          for name, filename in [('Body', 'DejaVuSans.ttf'),
079                                 ('Bold', 'DejaVuSans-Bold.ttf'),
080                                 ('Mono', 'DejaVuSansMono.ttf')]:
081              pdfmetrics.registerFont(TTFont(name, str(fontdir/filename)))
082          self.c = canvas.Canvas(str(dest), pagesize=A4)
083          self.c.setTitle('Filho paga para o Pai - análise de candidatos a CPFP')
084          self.c.setAuthor('Lucas Panonko Assis Belchior')
085          self.page, self.y = 0, 0
086
087      def new(self, title):
088          if self.page:
089              self.c.showPage()
090          self.page += 1
091          c = self.c
092          c.setFillColor(colors.HexColor(TEAL)); c.rect(M, H-35, CW, 3, fill=1, stroke=0)
093          c.setFont('Body', 8); c.setFillColor(colors.HexColor(GRAY))
094          c.drawString(M, H-51, 'TÓPICOS ESPECIAIS DE SEGURANÇA  |  P1  |  02/11/2026')
095          c.drawRightString(W-M, 25, str(self.page))
096          c.drawString(M, 25, 'Lucas Belchior  |  Filho paga para o Pai')
097          self.y = H-78
098          self.p(title, 19, 'Bold', 25)
099          self.y -= 6
100
101      def p(self, text, size=9.6, font='Body', leading=None):
102          para = Paragraph(text, ParagraphStyle('body', fontName=font, fontSize=size,
103                           leading=leading or size*1.42, textColor=colors.HexColor(INK)))
104          _, height = para.wrap(CW, H)
105          if self.y-height < 45:
106              raise ValueError(f'Texto ultrapassa a página {self.page}: {text[:60]}')
107          para.drawOn(self.c, M, self.y-height)
108          self.y -= height+9
109
110      def table(self, rows, widths=None, size=9):
111          style = ParagraphStyle('cell', fontName='Body', fontSize=size,
112                                 leading=size*1.35, textColor=colors.HexColor(INK))
113          cells = [[Paragraph(escape(str(x)), style) for x in row] for row in rows]
114          t = Table(cells, colWidths=widths or [CW/len(rows[0])]*len(rows[0]))
115          t.setStyle(TableStyle([
116              ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#e7f0ed')),
117              ('VALIGN', (0, 0), (-1, -1), 'TOP'),
118              ('LINEBELOW', (0, 0), (-1, 0), .5, colors.HexColor(TEAL)),
119              ('LINEBELOW', (0, 1), (-1, -1), .25, colors.HexColor('#dce1e3')),
120              ('LEFTPADDING', (0, 0), (-1, -1), 7),
121              ('RIGHTPADDING', (0, 0), (-1, -1), 7),
122              ('TOPPADDING', (0, 0), (-1, -1), 6),
123              ('BOTTOMPADDING', (0, 0), (-1, -1), 6),
124          ]))
125          _, height = t.wrap(CW, H)
126          if self.y-height < 45: raise ValueError(f'Tabela ultrapassa página {self.page}')
127          t.drawOn(self.c, M, self.y-height); self.y -= height+12
128
129      def image(self, path, height):
130          if self.y-height < 45: raise ValueError('Figura ultrapassa página')
```

Apêndice | build_report.py

```
        self.c.drawImage(str(path), M, self.y-height, CW, height,
                         preserveAspectRatio=True, anchor='c', mask='auto')
        self.y -= height+8


def build_report(results: Path, output: Path) -> Path:
    s = json.loads((results/'summary.json').read_text(encoding='utf-8'))
    plots(results, s)
    output.parent.mkdir(parents=True, exist_ok=True)
    r = Report(output)
    r.new('Filho paga para o Pai')
    r.p('Lucas Panonko Assis Belchior - Matrícula 12311BCC068', 10, 'Bold')
    r.p('Detecção e análise de candidatos a Child Pays for Parent (CPFP)', 12)
    r.p('1. Objetivo e dados', 12, 'Bold')
    r.p('O objetivo é detectar uma transação pai de baixa taxa e uma transação filho '
        'que consome pelo menos uma saída do pai, paga uma taxa maior e aparece no '
        'mesmo bloco. As taxas das duas transações são destinadas ao minerador que '
        'produziu esse bloco. A classificação econômica é uma heurística: a inclusão '
        'conjunta comprova a dependência, mas não a intenção de acelerar a confirmação [1, 2].')
    r.table([['Característica da amostra', 'Resultado'],
             ['Fonte', 'blocos.tgz fornecido nos exercícios da disciplina'],
             ['Arquivo compactado', n(s['archive_bytes'])+' bytes'],
             ['Blocos analisados', n(s['blocks'])],
             ['Alturas mínima e máxima', f"{s['height_min']} a {s['height_max']}"],
             ['Período dos timestamps (UTC)', s['start_utc']+' até '+s['end_utc']],
             ['Transações normais / coinbase', n(s['normal_transactions'])+' / '+n(s['coinbase'])],
             ['Anomalias de integridade detectadas', n(s['anomalies'])]], [CW*.42, CW*.58])
    r.p('A amostra não é tratada como uma sequência contínua de blocos nem como '
        'representativa de toda a rede. Os JSONs são lidos diretamente do TAR '
        'compactado, um bloco por vez. O hash do arquivo e os comandos de reprodução '
        'estão na página de referências.')
    r.p(f"Resultado principal: {n(s['candidates'])} pares candidatos, em "
        f"{n(s['blocks_with_candidates'])} dos {n(s['blocks'])} blocos.", 12, 'Bold')

    r.new('Como o algoritmo detecta os pares')
    r.p('2. Dependência e unidade de análise', 12, 'Bold')
    r.p('Cada saída é identificada por (tx_index, n), no formato fornecido. O '
        'tx_index é um identificador do provedor; o hash da transação é conservado '
        'para auditoria. Uma entrada do filho deve referenciar uma saída existente '
        'do pai, com o mesmo valor, e o pai deve aparecer antes do filho no bloco. '
        'Várias saídas consumidas do mesmo pai formam uma única aresta.')
    r.p('Construímos um grafo por bloco. A análise econômica usa somente componentes '
        'conexos com duas transações e uma aresta: um par simples e isolado. '
        'Componentes com três ou mais transações são contados separadamente. Isso '
        'evita atribuir a um par a taxa de um pacote que contém outros ancestrais.')
    r.p('3. Taxas e regra principal', 12, 'Bold')
    r.p('f = soma das entradas - soma das saídas (satoshis)<br/>'
        'v = teto(peso / 4) (vbytes); r = f / v (sat/vB)<br/>'
        'r_conjunto = (f_pai + f_filho) / (v_pai + v_filho)', 10, 'Mono')
    r.p('M é a mediana de sat/vB das transações normais do próprio bloco. '
        'Um par simples é candidato quando atende simultaneamente:')
    r.table([['Condição', 'Interpretação'],
             ['r_pai < M', 'Pai relativamente barato no bloco'],
             ['f_filho > f_pai', 'Filho paga mais em taxa absoluta'],
             ['r_filho > r_pai e r_filho >= 2 × r_pai', 'Filho tem maior taxa por tamanho'],
             ['r_conjunto >= M', 'Conjunto alcança a referência adotada']], [CW*.52, CW*.48])
    r.p('Pai com taxa zero é permitido; o filho deve pagar taxa positiva. Não se '
        'calcula razão dividindo por zero. Se M = 0, nenhum par atende à primeira '
        'condição. M não é o limite de aceitação do minerador: é uma escolha didática.')
    r.p('Fluxo: ler e validar bloco → indexar transações/saídas → criar arestas → '
        'identificar componentes → aplicar regra → agregar resultados. O grafo '
        'custa O(T + I + O), em média, por bloco; a mediana e as ordenações acrescentam '
        'custo. T, I e O representam transações, entradas e saídas.')

    r.new('Frequência e comparação das taxas')
```

Apêndice | build_report.py

```
196     r.table([['Indicador', 'Quantidade'],
197              ['Dependências diretas no mesmo bloco', n(s['direct_dependencies'])],
198              ['Pares simples elegíveis', n(s['simple_pairs'])],
199              ['Componentes complexos / transações nesses componentes',
200               n(s['complex_components'])+' / '+n(s['complex_transactions'])],
201              ['Candidatos / pares simples', n(s['candidates'])+' / '+n(s['simple_pairs'])+' =
    '+pct(s['candidate_fraction_pairs'])],
202              ['Filhos candidatos / transações normais', n(s['unique_children'])+' /
    '+n(s['normal_transactions'])+' = '+pct(s['candidate_fraction_transactions'])]], [CW*.69, CW*.31],
    8.6)
203     r.image(results/'figures/frequency.png', 170)
204     r.p('Figura 1. Candidatos por bloco amostrado; os intervalos sem blocos no arquivo '
205         'não são preenchidos com zeros.', 8.4)
206     r.image(results/'figures/rates.png', 190)
207     r.p('Figura 2. Todos os pares simples. Escala symlog nos dois eixos: linear '
208         'até 1 sat/vB e logarítmica acima, preservando os casos de taxa zero. '
209         'Os pontos destacados satisfazem todas as condições, não apenas a diferença de taxas.',
    8.4)
210
211     r.new('Taxas, valores e endereços')
212     ms = s['metrics']
213     r.table([['Medida entre candidatos', 'Mínimo', 'Mediana', 'Máximo']]+[
214         [label, n(ms[key]['minimum'], digits), n(ms[key]['median'], max(digits, 1)),
215          n(ms[key]['maximum'], digits)] for key, label, digits in [
216             ('parent_fee', 'Taxa do pai (sat)', 0), ('child_fee', 'Taxa do filho (sat)', 0),
217             ('parent_rate', 'Pai (sat/vB)', 3), ('child_rate', 'Filho (sat/vB)', 3),
218             ('package_rate', 'Conjunto (sat/vB)', 3),
219             ('consumed_sat', 'Valor consumido do pai (sat)', 0)]], [CW*.43, CW*.19, CW*.19,
    CW*.19], 8.3)
220     r.p(f"Os candidatos têm {n(s['unique_parents'])} pais e {n(s['unique_children'])} filhos "
221         f"distintos; {n(s['zero_fee_parents'])} pais pagam taxa zero. As taxas das "
222         f"transações únicas somam {n(s['unique_fee_total_sat'])} sat. O valor das "
223         f"saídas dos pais consumidas pelos filhos soma {n(ms['consumed_sat']['total'])} "
224         f"sat ({n(ms['consumed_sat']['total']/100000000, 8)} BTC). "
225         'Esse valor pode circular em outras transações: não representa dinheiro novo ou prejuízo.')
226     r.p(f"Foram observados {n(s['unique_addresses'])} endereços distintos nas saídas "
227         f"consumidas dos pais; {n(s['missing_address_outputs'])} de "
228         f"{n(s['consumed_outputs'])} saídas não possuem endereço informado.")
229     r.table([['Endereço da saída do pai', 'Pares', 'Valor (sat)']]+[
230         [a['address'], n(a['pairs']), n(a['value_sat'])] for a in s['top_addresses']],
231         [CW*.68, CW*.10, CW*.22], 7.8)
232     r.p('Tabela de até cinco endereços mais recorrentes. Um endereço é contado uma '
233         'vez por par, mesmo se houver várias saídas consumidas. Os valores agregam '
234         'essas saídas. Endereço não equivale a pessoa; a análise não identifica '
235         'proprietários nem deduz que pai e filho pertencem à mesma carteira.')
236
237     r.new('Exemplos rastreáveis nos dados')
238     for i, p in enumerate(s['examples'], 1):
239         r.p(f"Exemplo {i} - bloco {p['height']}", 11, 'Bold')
240         r.p('Pai: '+p['parent']+'<br/>Filho: '+p['child'], 7.2, 'Mono', 11)
241         r.table([['Medida', 'Pai', 'Filho'],
242                  ['Taxa (sat)', n(p['parent_fee']), n(p['child_fee'])],
243                  ['Tamanho virtual (vB)', n(p['parent_vsize']), n(p['child_vsize'])],
244                  ['Taxa por tamanho (sat/vB)', n(p['parent_rate'], 4), n(p['child_rate'], 4)]],
245                 [CW*.5, CW*.25, CW*.25], 8.5)
246         r.p(f"Taxa conjunta: {n(p['package_rate'], 4)} sat/vB; mediana do bloco: "
247             f"{n(p['median_rate'], 4)} sat/vB. Aumento absoluto em relação ao pai: "
248             f"{n(p['uplift'], 4)} sat/vB. Valor consumido: {n(p['consumed_sat'])} sat. "
249             f"Classificação: {'candidato' if p['candidate_2.0'] else 'rejeitado: '+p['reasons']}.",
     9)
250         consumed = '; '.join(f"saída {o['n']}: {n(o['value'])} sat, "
251                              f"{o['address'] or 'endereço ausente'}" for o in
    p['consumed_outputs'])
252         r.p(escape(consumed), 7.5)
253     r.p('Seleção dos exemplos: maior aumento absoluto da taxa do conjunto em relação '
```

Apêndice | build_report.py

```
        'ao pai e, depois, menor taxa do pai entre os demais candidatos. Os vínculos '
        'e cálculos foram reconferidos diretamente nos blocos originais. '
        'A seleção ilustra o mecanismo e não representa o caso típico.', 8.7)

    r.new('Sensibilidade, limites e conclusão')
    r.image(results/'figures/sensitivity.png', 185)
    r.p('Figura 3. Variação apenas do multiplicador mínimo da taxa por tamanho do '
        'filho em relação ao pai. Todas as demais condições permanecem fixas. '
        'Os três valores foram definidos antes de examinar os resultados.', 8.6)
    r.p('Limitações', 12, 'Bold')
    r.p('A regra usa taxas relativas ao próprio bloco, sem estimar a política '
        'real do minerador. Não há dados históricos da mempool, tempo de espera, '
        'declaração de intenção ou rótulos verdadeiros. Por isso, não se calculam '
        'acurácia, precisão ou recall, e os pares classificados são candidatos '
        'compatíveis com CPFP, não ocorrências confirmadas por intenção.')
    r.p('Pacotes com três ou mais transações ficam fora da classificação principal, '
        'embora possam conter CPFP. A contagem é restrita aos pares simples da '
        'amostra e não mede a frequência global da técnica. Gastos comuns podem '
        'satisfazer os critérios. Endereços ausentes limitam a análise de recorrência.')
    r.p('Validação', 12, 'Bold')
    r.p('Os testes sintéticos verificam: taxa zero, mediana zero, arredondamento de '
        'peso, taxas inconsistentes, entradas repetidas do mesmo pai, pais externos, '
        'cadeias, ramificações, coinbase, referências internas inválidas e endereço '
        'ausente. Na base completa, foram conferidos duplicados, taxas, referências '
        'internas e a ordem pai-filho. Os dois exemplos foram recalculados na fonte.')
    r.p('Conclusão', 12, 'Bold')
    r.p(f"O algoritmo encontrou {n(s['candidates'])} candidatos entre "
        f"{n(s['simple_pairs'])} pares simples ({pct(s['candidate_fraction_pairs'])}). "
        'O trabalho mostra como o grafo de UTxOs e a taxa conjunta permitem '
        'detectar um padrão econômico explicável. A principal distinção aprendida '
        'é entre observar uma dependência na cadeia e comprovar a motivação de quem '
        'criou a transação.')

    r.new('Reprodução e referências')
    r.p('Código e resultados', 12, 'Bold')
    r.p(f'<link href="{REPO}" color="{TEAL}">{REPO}</link>', 8.5)
    r.p('O repositório contém os dois scripts, testes, tabelas resumidas e este PDF. '
        'O arquivo original não é redistribuído: usar o blocos.tgz fornecido pela '
        'disciplina. O apêndice traz o código integral, para que o documento seja '
        'autossuficiente. As tabelas completas são regeneradas pelos comandos abaixo.')
    r.p('Python 3.12 ou superior. Dentro da pasta do repositório, com um interpretador '
        'Python instalado e funcional:')
    r.p('Dependências usadas: matplotlib 3.10.7, reportlab 4.4.9 e pypdf 6.10.0 '
        '(conteúdo de requirements.txt). O detector usa somente a biblioteca padrão.', 8.7)
    r.p('python -m pip install -r requirements.txt<br/>'
        'python -m unittest -v test_analyze<br/>'
        'python analyze.py --archive ../blocos.tgz --output results<br/>'
        'python build_report.py --results results<br/>'
        'No último comando, o PDF é gravado em report/.', 8, 'Mono', 13)
    r.p('Identificação exata do arquivo de entrada (SHA-256):', 9)
    r.p(s['archive_sha256'], 7.7, 'Mono')
    r.p('Saídas: pairs.csv (pares simples e motivos de rejeição), blocks.csv (censo '
        'por bloco), addresses.csv (endereços dos candidatos) e summary.json '
        '(parâmetros, totais, métricas e exemplos). Valores monetários estão em '
        'satoshis; 1 BTC = 100.000.000 sat.')
    r.p('Referências', 12, 'Bold')
    r.p('[1] Enunciado da atividade “(P1) Filho paga para o Pai”, disciplina de '
        'Tópicos Especiais de Segurança. Material fornecido ao aluno, 2026.<br/><br/>'
        '[2] BITCOIN CORE. Bitcoin Core 0.13.0 Released!, seção “More intelligent '
        'transaction selection for mining”, 23 ago. 2016. Disponível em: '
        '<link href="https://bitcoincore.org/en/2016/08/23/release-0.13.0/">'
        'https://bitcoincore.org/en/2016/08/23/release-0.13.0/</link>. '
        'Acesso em: 2 nov. 2026.<br/><br/>'
        '[3] blocos.tgz. Conjunto de blocos JSON fornecido nos exercícios anteriores '
        'da disciplina. Arquivo identificado pelo SHA-256 acima.', 9)
```

Apêndice | build_report.py

```
319      r.p('Apêndice - código-fonte integral', 12, 'Bold')
320      r.p('As páginas seguintes reproduzem analyze.py e build_report.py com números '
321          'de linha. Linhas longas são quebradas visualmente; para execução, usar '
322          'os arquivos do repositório. A lógica de detecção não depende de rede, '
323          'serviços pagos ou aprendizado de máquina.')
324      for filename in ['analyze.py', 'build_report.py']:
325          lines = []
326          for num, line in enumerate((ROOT/filename).read_text(encoding='utf-8').splitlines(), 1):
327              wrapped = textwrap.wrap(line.expandtabs(4), width=99, replace_whitespace=False,
328                                      drop_whitespace=False) or ['']
329              lines.extend([f'{num:03d}  '+wrapped[0]]+['     '+v for v in wrapped[1:]])
330          for offset in range(0, len(lines), 65):
331              r.new(f'Apêndice | {filename}')
332              r.c.setFont('Mono', 7.5)
333              r.c.setFillColor(colors.HexColor(INK))
334              for line in lines[offset:offset+65]:
335                  r.c.drawString(M, r.y, line)
336                  r.y -= 10.1
337      r.c.save()
338      print(f'PDF: {output.resolve()} ({r.page} páginas)')
339      return output
340
341
342  if __name__ == '__main__':
343      parser = argparse.ArgumentParser(description=__doc__)
344      parser.add_argument('--results', type=Path, default=Path('results'))
345      parser.add_argument('--output', type=Path,
346          default=Path('report/Filho_Paga_Para_Pai_Lucas_Belchior.pdf'))
347      args = parser.parse_args()
348      build_report(args.results, args.output)
```